

## 一、总体系统架构

平台采用前后端分离架构，确保计算逻辑与表现层解耦，支持跨端部署。

- **前端渲染层 (Web & 3D)**: 采用 Vue 3 + TypeScript 框架。2D UI 组件库选用 Element Plus 或 Ant Design Vue，保障表单重负载场景下的性能；3D 渲染引擎选用 Three.js，支持轻量化且高度定制化的 WebGL 图形展示。
- **桌面端容器 (客户端套壳)**: 选用 Tauri 或 Electron。Tauri 基于 Rust 开发，打包体积更小、内存占用更低，符合轻量化客户端需求；Electron 生态更为成熟。两者均可完美封装 HTML5 前端页面。
- **后端 API 服务层**: 采用 Python FastAPI 框架。由于底层计算模型已完全由 Python 函数构建，FastAPI 可实现原生无缝集成。FastAPI 具备极高的并发性能，且自动生成 OpenAPI (Swagger) 接口文档，直接满足向第三方输出 POST API 请求的要求。

## 二、核心模块技术实现

### 1. API 接口与数据流转

- **请求规范**: 计算请求统一通过 `POST /api/v1/calculate/drainage` 触发。
- **报文结构**: 采用 JSON 格式。请求体包含 `tunnel_type` (单洞/双洞) 及对应的参数字典。
- **第三方调用**: 通过标准 RESTful API 及 Bearer Token 鉴权，支持外部系统（如综合管廊平台、BIM 平台）直接 POST 传参并获取间距、孔径等优化布置结果。

### 2. 参数输入与 UI 交互设计

针对前端 33/34 个高密度参数，核心设计逻辑为“分区分级、逐步披露”，降低用户的认知负荷。

- **分模块折叠面板 (Accordion)**: 将参数按工程逻辑物理隔离。例如分为“地质水文参数”、“结构尺寸参数”、“材料属性参数”。
- **参数分级与默认值封装**:
  - **核心输入区**: 直接展示 15/16 个核心输入参数及 6 个复选参数。
  - **高级设置区**: 将其余 12 个默认参数折叠隐藏，默认填入规范推荐值或经验值。用户如需修改，展开“高级设置”面板即可干预。
- **典型案例库 (一键赋值)**: 开发“预设模板”功能。内置如“标准岩溶发育区双洞”、“高水压富水断层单洞”等工程案例。用户下拉选择对应案例后，表单 33 个参数瞬间完成自动赋值。
- **批量数据 IO**: 依托前端 `xlsx` 库或后端 Pandas 解析，支持下载标准 Excel 模板，用户在本地图表填好参数后一键导入；同样支持将当前表单的参数组合导出为 `.json` 或 `.xlsx` 存档。

### 3. 3D 模型构建与可视化交互

3D 模块的核心是参数化生成与数据双向绑定，无需导入外部庞大的 BIM 模型。

- **低参数隧道构建 (逐环生成)**:
  - 在 Three.js 中定义基础“计算单元（一环）”的几何体（如圆柱壳体或马蹄形截面）。
  - 基于传入的隧道总长度和单环长度，使用 `THREE.InstancedMesh` (实例化网格) 技术进行线性阵列复制。这能确保在生成数百环隧道时，依然保持 60FPS 的渲染帧率。
- **3D 交互控制 (旋转/剖切/隐藏)**:
  - **旋转与缩放**: 引入 `OrbitControls` 实现鼠标拖拽旋转、滚轮缩放。

- **剖切功能**：利用 Three.js 的 **ClippingPlanes** 特性，设置横向或纵向的剪裁平面，通过 UI 滑块实时调整剖切深度，查看隧道内部。
- **局部隐藏**：将隧道衬砌、初支、围岩、排水管分为不同的 **Group** 或 **Layer**。通过 UI 树形图（图层管理器）控制各图层的 **visible** 属性，实现局部隐藏。
- **计算结果映射与拾取 (参数反显)**：
  - 后端计算返回间距、孔径数据后，前端根据这些数据，在 3D 空间内的特定坐标点生成排水管网格模型。
  - 引入 **Raycaster** (射线拾取)。用户在 3D 模型中点击某段排水管时，高亮该管段，并在鼠标位置弹出 HTML 浮窗 (Tooltip)，精准显示该管段的“间距：XX m，孔径：XX mm”。
- **多方案 3D 对比机制**：
  - 支持“分屏对比 (Split Screen)”模式。在一个 WebGL Canvas 中设置两个独立的 Camera 和 Viewport，左侧显示方案 A 的 3D 结果，右侧显示方案 B 的结果。
  - 两侧视角支持联动（同步旋转/缩放），直观暴露不同计算参数下，排水管网空间布置密度的视觉差异。

三、工程文件目录架构

基于前述技术架构方案（FastAPI 后端 + Vue3/Three.js 前端 + Tauri 桌面端套壳），以下是标准化工程文件目录结构：

|                                          |                                     |
|------------------------------------------|-------------------------------------|
| tunnel-drainage-platform/                | # 工程根目录                             |
| ├─ api-server/                           | # 后端 API 服务层 (Python FastAPI)       |
| │   │   └─ app/                          |                                     |
| │   │       │   └─ main.py               | # FastAPI 应用入口                      |
| │   │       │   └─ api/                  | # 路由层：POST API 接口定义                 |
| │   │   (v1/calculate/drainage)          |                                     |
| │   │       │   └─ core/                 | # 核心配置：CORS、中间件、日志                  |
| │   │       │   └─ schemas/              | # 数据验证模型 (Pydantic)：定义单洞/双洞的        |
| │   │   33/34 个输入参数结构                    |                                     |
| │   │       │   └─ services/             | # 业务逻辑层：处理前端请求并调用底层计算模型             |
| │   │       │   └─ core_calc/            | # 核心计算逻辑 (已有的 Python 函数库)           |
| │   │           │   └─ __init__.py       |                                     |
| │   │           │   └─ single_tube.py    | # 单洞智能排水计算模型                        |
| │   │           │   └─ double_tube.py    | # 双洞智能排水计算模型                        |
| │   │           │   └─ constants.py      | # 默认参数/规范经验值 (12个默认参数配置)            |
| │   │       │   └─ tests/                | # 后端单元测试                            |
| │   │       │   └─ requirements.txt      | # Python 依赖清单                       |
| │   │       └─ Dockerfile                | # 后端容器化部署文件                         |
| ├─ web-ui/                               | # 前端渲染层 (Vue 3 + TypeScript + Vite) |
| │   │   └─ public/                       |                                     |
| │   │       └─ templates/                | # 静态资源：提供参数批量导入的标准 Excel 模板         |
| │   下载                                   |                                     |
| │   │   └─ src/                          |                                     |
| │   │       │   └─ api/                  | # 前端接口封装 (Axios)：对接 FastAPI         |
| │   │       │   └─ assets/               | # 样式与图片资源                           |
| │   │       │   └─ components/           |                                     |
| │   │           │   └─ ParameterForm.vue | # UI 组件：参数输入面板 (分区分级、折叠面板设计)        |
| │   │           │   └─ CaseSelector.vue  | # UI 组件：典型案例库下拉选择 (一键赋值)            |

|            |                           |                                |
|------------|---------------------------|--------------------------------|
| 能          | └─ ResultTable.vue        | # UI 组件：计算结果(间距、孔径)数据表与导出功     |
|            | └─ engine3d/              | # 3D 核心模块 (Three.js 独立封装)      |
|            | └─ TunnelInstancedMesh.ts | # 逐环隧道模型生成器 (低参数构建)            |
|            | └─ DrainagePipes.ts       | # 排水管网结果映射与渲染模块                |
|            | └─ SceneControls.ts       | # 交互控制器：旋转、缩放、拾取(Raycaster)反显  |
|            | └─ ClippingManager.ts     | # 剖切与局部隐藏(图层控制)逻辑              |
|            | └─ SplitScreenViewer.ts   | # 多方案 3D 对比分屏渲染逻辑              |
|            | └─ views/                 |                                |
|            | └─ SingleTubeView.vue     | # 页面：单洞计算工作台                   |
|            | └─ DoubleTubeView.vue     | # 页面：双洞计算工作台                   |
|            | └─ store/                 | # 状态管理 (Pinia)：管理 33/34 个参数的全局 |
| 响应状态       | └─ App.vue                | # 前端根组件                        |
|            | └─ main.ts                | # 前端入口文件                       |
|            | └─ package.json           | # Node.js 依赖清单                 |
|            | └─ vite.config.ts         | # 前端构建配置                       |
|            | └─ src-tauri/             | # 桌面端容器层 (Tauri 基于 Rust)       |
|            | └─ src/                   |                                |
|            | └─ main.rs                | # 桌面端原生窗口配置、系统托盘、系统级文件读写       |
| 权限声明       | └─ icons/                 | # 桌面客户端各分辨率图标                  |
|            | └─ tauri.conf.json        | # Tauri 核心打包配置文件 (定义窗口大小、套壳    |
| web-ui 目录) | └─ Cargo.toml             | # Rust 依赖清单                    |
|            | └─ docs/                  | # 项目文档                         |
|            | └─ api_docs/              | # 接口文档 (供第三方对接)                |
|            | └─ architecture.md        | # 架构设计说明                       |
|            | └─ .gitignore             | # Git 忽略配置                     |
|            | └─ README.md              | # 工程说明文件                       |